

# Arquitetura RAG para Gerador Automático de Prompts Adaptativos

## Arquitetura RAG Unificada para Geração Automática de Prompts Dinâmica e Adaptativa

### Introdução: Filosofema e Racionalidade

*Filosofema inicial:*

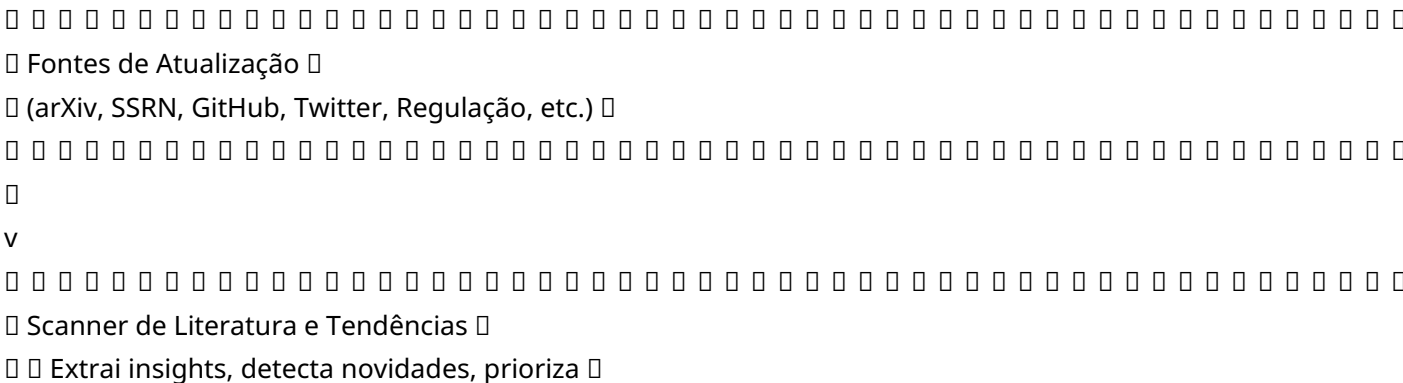
**"O melhor prompt é aquele que encapsula a alma do problema - e como toda alma, evolui, adapta-se e transcende."**

A engenharia de prompts, tradicionalmente vista como uma arte manual, torna-se, neste contexto, um **sistema vivo**: um ecossistema auto-regenerativo, capaz de se adaptar às mudanças do mercado, da ciência e da linguagem. O objetivo desta arquitetura é criar um **gerador automático de prompts**, unificando conceitos de embeddings de usuários, camadas de embedding, espaço latente, raciocínio vetorial e lógico, e geração de linguagem natural, para implementar o sistema descrito no documento "Engenharia de Prompts Dinâmica e Adaptativa". A seguir, apresentamos um outline essencialista, estruturado em seções hierárquicas, com emojis, filosofemas, rationales, metacríticas, sugestões e exemplos práticos, cobrindo desde a ingestão e compressão semântica de memória até a geração, refatoração e governança de prompts, com ciclos de feedback.

### 1. Visão Geral: Mandala Viva da Engenharia de Prompts

A arquitetura proposta é inspirada em sistemas vivos: **não é uma máquina, mas um jardim** - precisa ser regado, podado e, às vezes, deixado em pousio para renascer mais forte. O sistema é composto por módulos interconectados, cada um responsável por uma etapa do ciclo de vida do prompt, desde a ingestão de conhecimento até a refatoração baseada em feedback.

#### 1.1 Diagrama Macro do Sistema



v  
 Motor de Meta-Prompts Recursivos  
 Gera, revisa, otimiza e versiona prompts automaticamente  
 v v  
 Prompt Repository  
 Prompt Orchestrator  
 (catalog, versioning, (routers, chains, workflows)  
 metadata, tags)  
 v v  
 Prompt Discovery  
 Execution Environment  
 (query, recommendation, (LLMs, MCP, code assistants)  
 drift detection)  
 Feedback Loop (métricas de desempenho, logs, avaliações)

## 2. Componentes Essenciais da Arquitetura

### 2.1 Prompt Repository: Catálogo Vivo e Versionado

#### Função:

Centralizar, versionar e organizar todos os prompts, metadados, arquétipos e formulários, garantindo rastreabilidade, reuso e governança.

#### Estrutura de Dados (exemplo simplificado):

```

{
  "prompt_id": "risk-assessment-liquidity-2026",
  "version": "2.1.0",
  "title": "Avaliação de Risco de Liquidez em Protocolos DeFi",
  "archetype": "diagnóstico",

```

```

"tags": ["risco", "liquidez", "defi", "due-diligence"],
"author": "sistema-auto",
"created": "2026-02-20",
"updated": "2026-02-22",
"content": "Analise o protocolo {protocol} quanto aos seguintes riscos de liquidez...",
"parameters": ["protocol", "time_horizon"],
"dependencies": ["collector-defillama"],
"tests": [
{
"input": {"protocol": "aave", "time_horizon": "30d"},
"expected_output_contains": ["concentração", "pool depth"]
}
],
"metrics": {
"avg_success_rate": 0.92,
"avg_response_time": 2.3
},
"status": "active",
"changelog": [
{"version": "2.0.0", "date": "2026-02-15", "changes": "Adicionado suporte a novos protocolos"},
{"version": "2.1.0", "date": "2026-02-22", "changes": "Otimizado com DSPy"}
]
}

```

### Rationale:

Um repositório bem estruturado permite busca eficiente, reuso, rastreabilidade e governança, além de facilitar a automação de testes, métricas e auditoria.

---

## 2.2 Motor de Meta-Prompts Recursivos

### Função:

Gerar, revisar, otimizar e versionar prompts automaticamente, utilizando feedback, novas fontes de dados e padrões de sucesso.

- **Prompt Generator:** Cria novos prompts a partir de gaps, tendências e exemplos de sucesso.
- **Prompt Optimizer (ex: DSPy):** Ajusta prompts via otimização automática, testes A/B e fine-tuning.
- **Prompt Linter:** Verifica clareza, completude, consistência e aderência a boas práticas (ex: LLM Prompt Linter<sup>[1]</sup>).
- **Prompt Drift Detector:** Monitora perda de eficácia e sugere revisão.

### Filosofema:

*"Um prompt que não evolui é um oráculo que silencia."*

### Exemplo Prático:

Ao detectar queda na taxa de sucesso de um prompt, o sistema aciona o linter e sugere variações otimizadas, testando-as automaticamente.

---

## 2.3 Prompt Discovery, Recomendação e Playbooks

### Função:

Permitir busca semântica, recomendação personalizada e curadoria de coleções de prompts (playbooks) para diferentes perfis e contextos.

- **Query Semântica:** Busca por embeddings, tags ou similaridade.
- **Recomendação:** Sugere prompts com base em perfil, contexto e histórico de uso (user embeddings<sup>[3]</sup>).
- **Playbooks:** Coleções temáticas para cenários específicos (ex: "Bear Market", "Onboarding").

### Rationale:

A personalização aumenta a relevância e a eficácia dos prompts, adaptando-se ao perfil e ao momento do usuário.

---

## 2.4 Prompt Orchestrator: Roteamento, Chains e Workflows

### Função:

Orquestrar a execução de prompts, roteando consultas para o prompt mais adequado, compondo cadeias (chains) e workflows complexos.

- **Routers:** Direcionam queries para prompts ideais (ex: Prompt Orchestrator<sup>[4]</sup>).
- **Chains:** Sequências onde a saída de um prompt alimenta o próximo (ex: diagnóstico → recomendação → execução).
- **Workflows:** Orquestração com condições, loops e paralelismo.
- **Prompt Flare:** Prompts de curta duração disparados por eventos.

### Exemplo Prático:

Em um atendimento ao cliente, o router identifica o tipo de dúvida, aciona uma chain de prompts (diagnóstico → solução → follow-up) e registra métricas de cada etapa.

---

## 2.5 Prompt Evaluation, A/B Testing e Métricas

### Função:

Avaliar a qualidade dos prompts por meio de métricas objetivas e subjetivas, testes automatizados e feedback do usuário.

- **Métricas:** Clareza, taxa de sucesso, tempo de resposta, satisfação do usuário<sup>[6]</sup>.
- **A/B Testing:** Comparação de variações de prompts em produção.

- **Cognitive Prompting:** Avaliação do impacto no raciocínio do modelo (ex: cadeias de pensamento<sup>[8]</sup>).
- **Prompt Tests:** Testes automatizados para garantir robustez.

**Rationale:**

A avaliação contínua permite identificar rapidamente drifts, regressões e oportunidades de melhoria.

---

## 2.6 Prompt Governance: Versionamento, Controle de Acesso e Compliance

**Função:**

Garantir governança, rastreabilidade, compliance (LGPD/GDPR) e segurança dos prompts.

- **Versionamento:** Git com tags semânticas, changelog detalhado<sup>[10][11]</sup>.
- **Controle de Acesso:** Permissões para criação, edição e aprovação.
- **Processo de Revisão:** Pull requests e revisão por pares.
- **Depreciação:** Marcação de prompts obsoletos e migração assistida.
- **Compliance:** Adesão a políticas de privacidade e proteção de dados (LGPD/GDPR)<sup>[13]</sup>.

**Exemplo Prático:**

Ao atualizar um prompt sensível, o sistema exige revisão dupla e registra todas as alterações para auditoria.

---

## 2.7 Integração com Fontes Externas e Scanner de Literatura

**Função:**

Manter o sistema atualizado com o estado da arte, novas tendências e descobertas científicas.

- **Scanner de Literatura:** Monitora arXiv, SSRN, GitHub, Twitter, etc., extraindo insights relevantes.
- **Auto-Update:** Atualiza prompts automaticamente com base em novas informações.
- **Feedback Loop:** Métricas de uso e sucesso retroalimentam o motor de geração.

**Rationale:**

A atualização contínua evita obsolescência e garante alinhamento com as melhores práticas do mercado e da ciência.

---

## 2.8 Prompt Metacognitivo e Hermenêutico

**Função:**

Explorar e refinar o próprio processo de geração de prompts, promovendo reflexão, autocrítica e evolução simbólica.

- **Prompts de Metaconversa:** Perguntam ao modelo sobre seu próprio raciocínio.





Converter textos, perfis e contextos em vetores densos, reduzindo dimensionalidade e facilitando buscas semânticas eficientes<sup>[16]</sup>.

**Exemplo Prático:**

Documentos e queries são embutidos no mesmo espaço vetorial, permitindo matching semântico robusto.

---

### 4.3 Espaço Latente e Raciocínio Vetorial

**Função:**

Operar no espaço latente para comparar, combinar e manipular conceitos de forma matemática, promovendo raciocínio vetorial e lógico<sup>[18]</sup>.

**Exemplo Prático:**

Prompt  $\text{embedding}(\text{"rei"}) - \text{embedding}(\text{"homem"}) + \text{embedding}(\text{"mulher"}) \approx \text{embedding}(\text{"rainha"})$ .

---

### 4.4 Raciocínio Lógico e Cadeias de Pensamento (CoT)

**Função:**

Implementar prompts que incentivam o modelo a explicitar cadeias de raciocínio, promovendo transparência e robustez<sup>[8][19]</sup>.

**Exemplo Prático:**

Prompt: "Explique passo a passo como chegou à resposta."

Saída: "Primeiro, considere X; depois, avalie Y; por fim, concluí Z."

---

### 4.5 Geração de Linguagem Natural e Meta-Prompts Recursivos

**Função:**

Utilizar LLMs para gerar, refinar e analisar prompts, criando meta-prompts que automatizam a própria engenharia de prompts<sup>[21]</sup>.

**Exemplo Prático:**

Meta-prompt: "Gere um prompt para análise de risco de liquidez em DeFi, incluindo parâmetros e exemplos de saída."

---

### 4.6 Ingestão e Compressão Semântica de Memória

**Função:**

Ingerir grandes volumes de dados, comprimir semanticamente (ex: DragonMemory<sup>[23]</sup>), e armazenar em bancos vetoriais para recuperação eficiente.

**Exemplo Prático:**

Documentos longos são chunkados, embutidos e comprimidos em vetores latentes, reduzindo custo e latência de busca.

---

## 4.7 Ciclos de Feedback Conversacional e Governança

### **Função:**

Coletar feedback explícito e implícito dos usuários, métricas de desempenho e logs, retroalimentando o sistema para ajustes automáticos e governança<sup>[25]</sup>.

### **Exemplo Prático:**

Usuários avaliam respostas; prompts com baixa performance são automaticamente revisados ou depreciados.

---

## 4.8 Prompt Repository: Versionamento, Metadata e Arquétipos

### **Função:**

Manter histórico de versões, metadados detalhados e classificação por arquétipos, facilitando busca, auditoria e evolução<sup>[10][11]</sup>.

### **Exemplo Prático:**

Prompt "diagnóstico" v2.1.0, tags: #risco, #liquidez, status: ativo, changelog: "otimizado com DSPy".

---

## 4.9 Prompt Discovery, Recommendation e Playbooks

### **Função:**

Oferecer mecanismos de busca semântica, recomendação personalizada e playbooks temáticos para diferentes cenários e perfis.

### **Exemplo Prático:**

Usuário busca por "due diligence DeFi" e recebe recomendações de prompts e playbooks relevantes.

---

## 4.10 Prompt Orchestrator: Routers, Chains e Workflows

### **Função:**

Orquestrar a execução de prompts, compondo cadeias e workflows adaptativos, com roteamento inteligente e fallback.

### **Exemplo Prático:**

Workflow: Diagnóstico → Recomendação → Execução → Follow-up, com logs e métricas de cada etapa.

---

## 4.11 Prompt Evaluation, A/B Testing e Métricas

### **Função:**

Avaliar continuamente a eficácia dos prompts, realizando testes A/B, coleta de métricas e ajustes automáticos.

### **Exemplo Prático:**

Prompt A apresenta 92% de sucesso, Prompt B 85%; sistema promove Prompt A para produção.

---

## 4.12 Prompt Linting, Drift Detection e Testes Automatizados

### **Função:**

Aplicar linting automatizado, detectar drift de performance e executar testes automatizados para garantir robustez e aderência a padrões<sup>[26]</sup>.

### **Exemplo Prático:**

Prompt linter identifica ambiguidade; drift detector sugere revisão após queda de 10% na taxa de sucesso.

---

## 4.13 Integração com Fontes Externas e Scanners de Literatura

### **Função:**

Monitorar fontes externas, detectar tendências e alimentar o sistema com novos insights e dados.

### **Exemplo Prático:**

Scanner detecta novo padrão de prompt em repositório GitHub; sistema sugere adaptação automática.

---

## 4.14 Semantic Cache e Otimização de Performance

### **Função:**

Utilizar cache semântico para acelerar respostas a queries recorrentes, reduzindo custo e latência<sup>[27]</sup>.

### **Exemplo Prático:**

Perguntas similares são respondidas instantaneamente via cache semântico, sem reprocessamento completo.

---

## 4.15 MCP e Integração com Assistentes e Code Assistants

### **Função:**

Expor prompts como ferramentas para agentes MCP, code assistants e ambientes de desenvolvimento<sup>[29]</sup>.

### **Exemplo Prático:**

Prompt recomendado automaticamente no VS Code conforme contexto do código.

---

## 4.16 Segurança, Privacidade e Compliance (LGPD/GDPR)

### **Função:**

Garantir conformidade com LGPD/GDPR, controle de acesso, anonimização e rastreabilidade de dados e prompts<sup>[13]</sup>.

**Exemplo Prático:**

Prompt sensível exige dupla revisão e registro de alterações para auditoria.

---

## 4.17 Infraestrutura: Vector DBs, Orquestradores e MLOps

**Função:**

Utilizar bancos vetoriais escaláveis (Qdrant, Pinecone, Weaviate, Milvus<sup>[31]</sup>), orquestradores (LangChain, FastAPI, Airflow) e práticas de MLOps para garantir robustez, escalabilidade e integração contínua.

**Exemplo Prático:**

Documentos são chunkados, embutidos e armazenados em Qdrant; workflows orquestrados via LangChain.

---

## 5. Exemplo Prático: Fluxo de Criação e Refatoração de Prompts

### 5.1 Cenário: Diagnóstico de Hype em Ativos P2E

**Prompt Gerado Automaticamente:**

# Prompt: Diagnóstico de Hype em Ativos P2E

**## Objetivo**

Detectar se um ativo de jogos Web3 está em bolha especulativa, usando métricas de sentimento, mindshare e valuation.

**## Parâmetros**

- `token`: símbolo do token (ex: JEWEL)
- `mindshare\_data`: dataframe com menções diárias
- `price\_data`: histórico de preços
- `nvt\_data`: NVT ratio

**## Instruções**

1. Calcule o crescimento de mindshare nos últimos 7 dias.
2. Calcule a variação de preço no mesmo período.
3. Obtenha o NVT atual e compare com a média histórica.
4. Se  $\text{mindshare\_growth} > 30\%$  E  $\text{price\_growth} > 50\%$  E  $\text{NVT} > \text{média} + 2\sigma$ , retorne "ALERTA: Hype excessivo, risco de bolha".
5. Caso contrário, retorne "Sem sinais claros de bolha".

**## Exemplo de saída**

```
{  
  "status": "ALERTA",  
  "indicadores": {
```

```

"mindshare_growth": 0.45,
"price_growth": 0.60,
"nvt_zscore": 2.3
},
"recomendacao": "Reduzir exposição ou implementar hedge."
}

```

#### Fluxo Automatizado:

1. **Necessidade Detectada:** Scanner identifica aumento de buscas por "hype P2E".
2. **Geração:** Motor de meta-prompts cria prompt inicial.
3. **Otimização:** DSPy ajusta instruções e parâmetros.
4. **Validação:** Linter verifica clareza e completude.
5. **Registro:** Prompt é versionado e catalogado.
6. **Publicação:** Disponibilizado via API/MCP.
7. **Uso:** Usuários executam prompt; métricas são coletadas.
8. **Feedback:** Baixa taxa de sucesso aciona revisão automática.
9. **Evolução:** Prompt é refinado e ciclo reinicia.

## 6. Metacríticas, Sugestões e Filosofemas

| Aspecto                 | Metacrítica                                                                  | Sugestão                                                                            |
|-------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Complexidade</b>     | Sistema pode se tornar pesado se todos os módulos forem implementados juntos | Implementação incremental: comece pelo repositório, scanner e motor de meta-prompts |
| <b>Governança</b>       | Risco de prompts desatualizados sem revisão humana                           | Combinar revisão automática (linter) com revisão humana (pull requests, comitês)    |
| <b>Drift</b>            | Difícil medir sem feedback explícito                                         | Usar proxies como taxa de sucesso, feedback implícito (cliques, tempo de uso)       |
| <b>Integração</b>       | MCP e code assistants exigem conhecimento técnico                            | Fornecer scripts prontos, documentação e exemplos                                   |
| <b>Prompt como Arte</b> | Subjetivo, difícil de automatizar                                            | Espaço para curadoria humana, competições de design, galerias de prompts artísticos |
| <b>Auto-geração</b>     | Pode gerar prompts genéricos ou redundantes                                  | Usar exemplos de sucesso e fine-tuning com dados reais de interação                 |

*Filosofema final:*

**"Um sistema de prompts não é uma máquina, mas um jardim - precisa ser regado, podado, e às vezes deixado em pousio para renascer mais forte."**

---

## 7. Conclusão: Essencialismo, Fluidez e Futuro

A arquitetura RAG unificada aqui proposta transforma a engenharia de prompts de uma atividade artesanal em um **ecossistema auto-regenerativo**, essencialista e pragmático. Ela integra user embeddings, camadas de embedding, espaço latente, raciocínio vetorial e lógico, e geração de linguagem natural, promovendo ciclos contínuos de feedback, governança e evolução.

### Principais Takeaways:

- **Organização e versionamento** são a base para rastreabilidade e evolução.
- **Automação e feedback** garantem adaptação contínua e redução de drift.
- **Personalização via embeddings** aumenta relevância e eficácia.
- **Governança e compliance** asseguram segurança, privacidade e confiança.
- **Integração com agentes, MCP e code assistants** amplia o alcance e a utilidade dos prompts.
- **Espaço para arte, ludolinguagem e metacognição** mantém o sistema humano, criativo e adaptativo.

### Próximos Passos:

1. Implementar o repositório de prompts e scanner de literatura.
2. Integrar motor de meta-prompts e linter automatizado.
3. Orquestrar chains e workflows básicos.
4. Coletar métricas e iniciar ciclos de feedback.
5. Expandir para integração com MCP, code assistants e módulos artísticos.

---

## Outline Resumido (com Emojis)

- Introdução: Filosofema e racionalidade
- Visão Geral: Mandala viva da engenharia de prompts
- Componentes essenciais:
  - Prompt Repository
  - Motor de Meta-Prompts Recursivos
  - Prompt Discovery, Recommendation e Playbooks
  - Prompt Orchestrator
  - Prompt Evaluation e Métricas
  - Prompt Governance
  - Integração com fontes externas

- Prompt Metacognitivo
- Aesthetic Prompt Design
- Integração com AI Assistants e MCP
- Ciclo de Vida do Prompt
- Integração dos conceitos-chave (user embeddings, embeddings, espaço latente, CoT, etc.)
- Exemplo prático: fluxo de criação e refatoração
- Metacríticas, sugestões e filosofemas
- Conclusão: Essencialismo, fluidez e futuro

---

**Este outline serve como blueprint para implementação de sistemas RAG unificados, dinâmicos e adaptativos, capazes de evoluir com o tempo, aprender com o uso e manter-se alinhados ao estado da arte e às necessidades dos usuários.**

---

## References (31)

1. *GitHub - abshingate/llm-prompt-linter: A comprehensive framework for ....*  
<https://github.com/abshingate/llm-prompt-linter>
2. *USER-LLM: Efficient LLM contextualization with user embeddings.*  
<https://research.google/blog/user-llm-efficient-llm-contextualization-with-user-embeddings/>
3. *labgadget015-dotcom/prompt-orchestrator - GitHub.* <https://github.com/labgadget015-dotcom/prompt-orchestrator>
4. *Prompt Engineering Evaluation Metrics: How to Measure Prompt Quality.*  
<https://www.leanware.co/insights/prompt-engineering-evaluation-metrics-how-to-measure-prompt-quality>
5. *Demystifying Long Chain-of-Thought Reasoning in LLMs.* <https://arxiv.org/abs/2502.03373>
6. *Managing and Versioning Prompts - apxml.com.* <https://apxml.com/courses/getting-started-with-llm-toolkit/chapter-2-advanced-prompting-techniques/prompt-management-and-versioning>
7. *11 Prompt Engineering Best Practices Every Modern Dev Needs.*  
<https://mirascope.com/blog/prompt-engineering-best-practices>
8. *Governança e boas práticas na Lei Geral de Proteção de Dados por meio ....*  
<https://pos.direito.ufmg.br/rbep/index.php/rbep/article/download/894/719/3397>
9. *Prompt-Based and Modality-Semantic Enhanced Multimodal Recommendation ....*  
[https://link.springer.com/chapter/10.1007/978-981-96-9812-7\\_5](https://link.springer.com/chapter/10.1007/978-981-96-9812-7_5)
10. *What Are Embedding Layers in Neural Networks? - Baeldung.*  
<https://www.baeldung.com/cs/neural-nets-embedding-layers>
11. *Advances in Generative AI Latent Space Reasoning: Comparing ... - LinkedIn.*

<https://www.linkedin.com/pulse/advances-generative-ai-latent-space-reasoning-comparing-lima-y6ulc>

12. *Prompt Engineering Best Practices: Tips, Tricks, and Tools.*

<https://www.digialocean.com/resources/articles/prompt-engineering-best-practices>

13. *The Definitive Guide to Meta-Prompting | createXflow.* <https://createxflow.com/meta-prompting-guide-architecture-implementation/>

14. *DragonMemory: Neural Sequence Compression for Production RAG.*

[https://dev.to/damjan\\_akelj\\_be1aab4a715/dragonmemory-neural-sequence-compression-for-production-rag-54b6](https://dev.to/damjan_akelj_be1aab4a715/dragonmemory-neural-sequence-compression-for-production-rag-54b6)

15. *retrieval\_with\_feedback\_loop.ipynb - Colab.*

[https://colab.research.google.com/github/NirDiamant/RAG\\_Techniques/blob/main/all\\_rag\\_techniques/retrieval\\_with\\_feedback\\_loop.ipynb](https://colab.research.google.com/github/NirDiamant/RAG_Techniques/blob/main/all_rag_techniques/retrieval_with_feedback_loop.ipynb)

16. *Prompt Linter - Visual Studio Marketplace.*

<https://marketplace.visualstudio.com/items?itemName=Ignire.prompt-linter>

17. *Semantic Cache: The Smartest Way to Speed Up RAG (Without More GPUs).*

<https://www.codebrains.co.in/blog/2025/ai/semantic-cache-smartest-way-to-speed-up-rag>

18. *MCP For Agent Integrations - A Practical Guide.* <https://blog.janaka.me/posts/mcp-agent-integration.html>

19. *Best Vector Databases in 2026: A Complete Comparison Guide.*

<https://www.firecrawl.dev/blog/best-vector-databases>